

Tooling for OCL

OCL plug-in and interpreter for Eclipse IDE

Ayoub CANON, Félix HORCHOLLE, Charlotte LADRIERE
Guilhem MANGENOT, Rémy SANGOI, Paul SOMSON

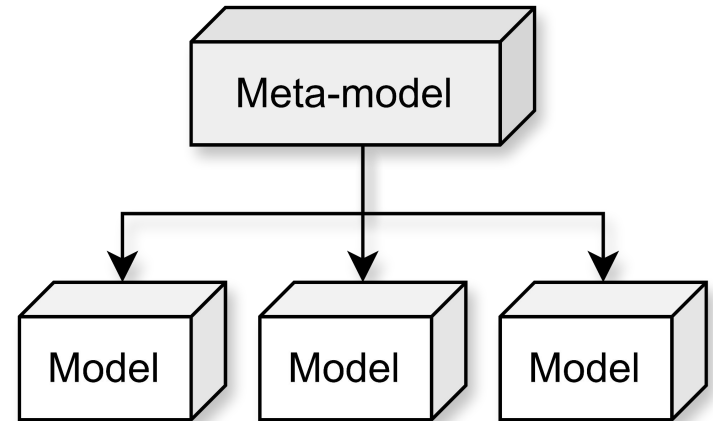
Supervisors : Xavier Cregut & Guillaume Dupont

What is Model Based Software Engineering ?

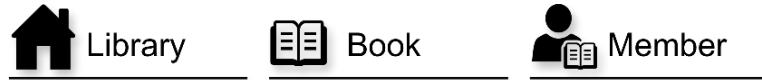
MBSE: Model Based Software Engineering

“The formalized application of **modeling** to support **system requirements, design, analysis, verification** and **validation** activities beginning in the conceptual design phase and continuing **throughout development and later life cycle phases**”

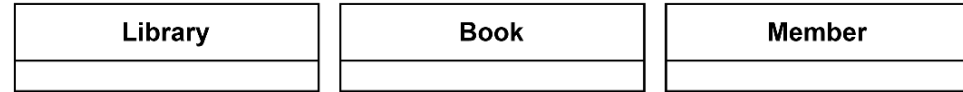
- INCOSE



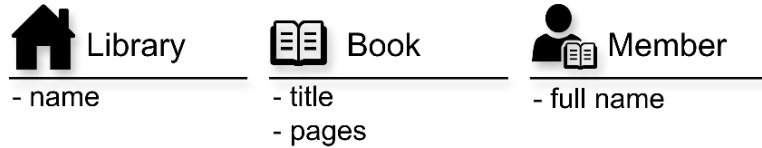
Meta-model example: library book lending system



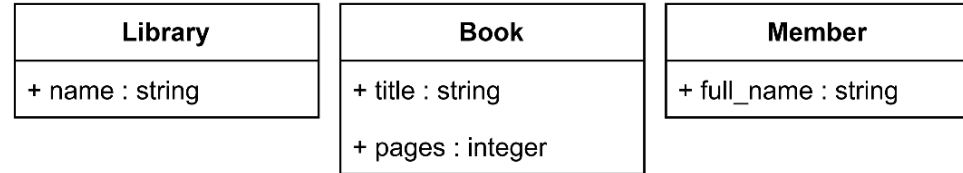
UML



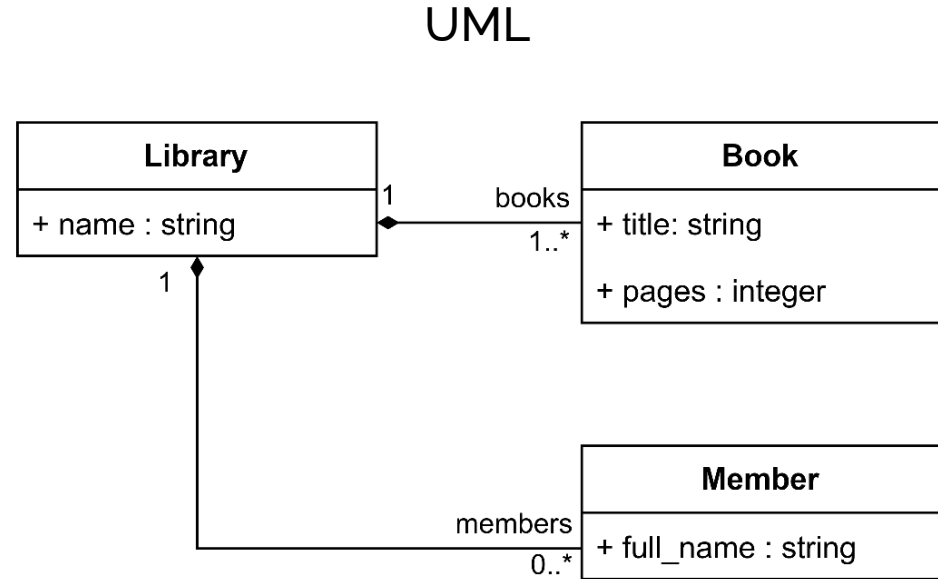
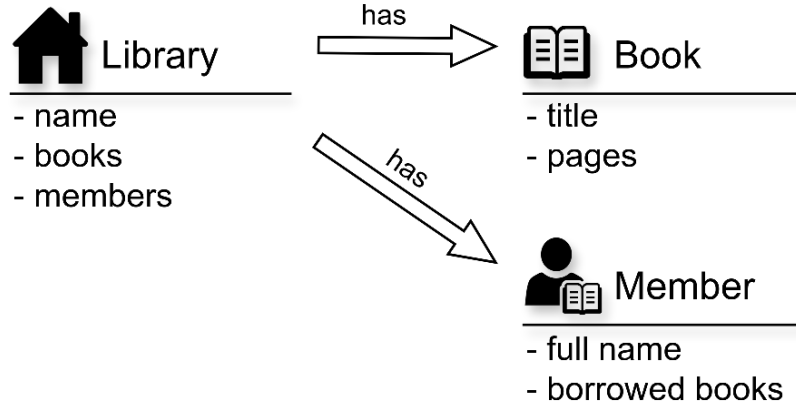
Meta-model example: library book lending system



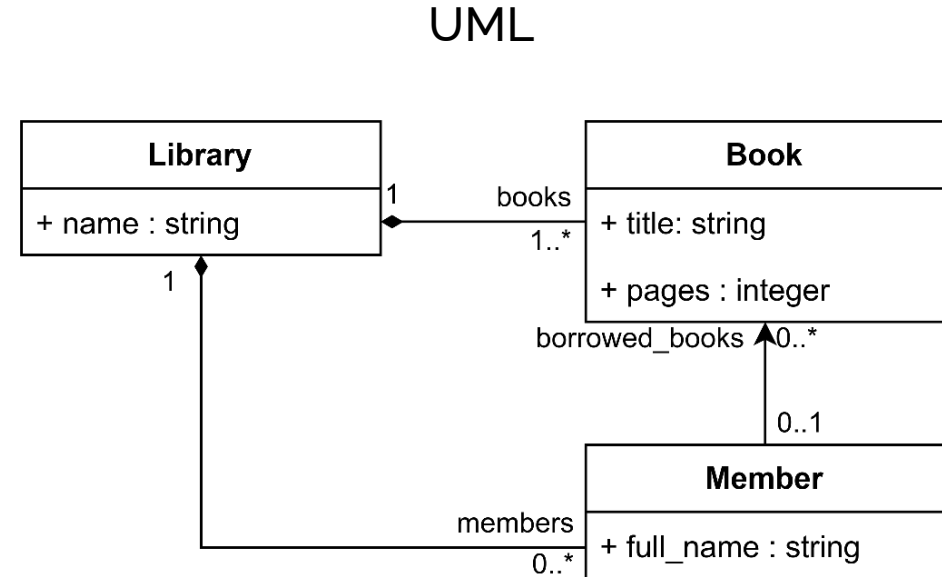
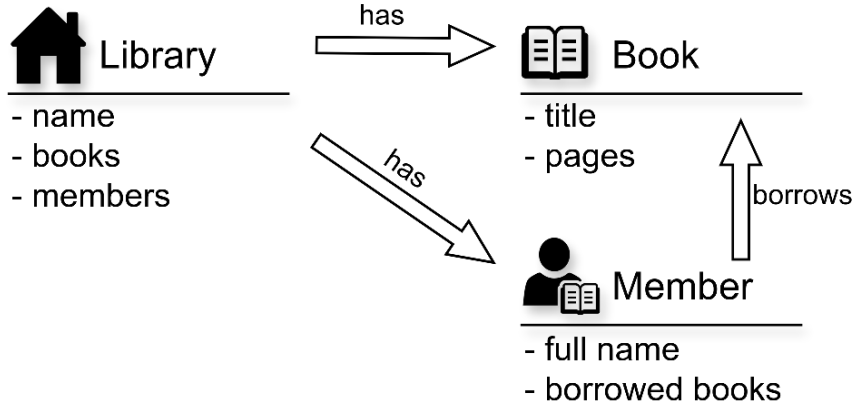
UML



Meta-model example: library book lending system



Meta-model example: library book lending system



Model example:



Smith's



The Hobbit

- pages = 336



The Little Prince

- pages = 96



The Alchemist

- pages = 192



Da Vinci Code

- pages = 400

borrowed

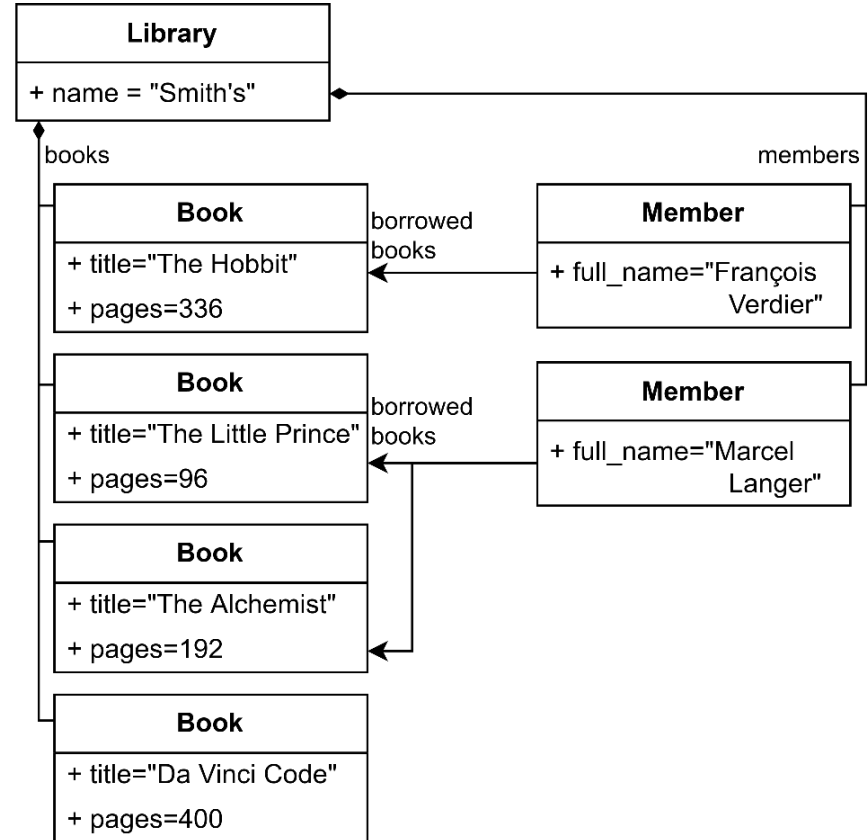


François Verdier

borrowed



Marcel Langer



Invalid model example:



Smith's



The Hobbit

- pages = 336



Weird Book

- pages = -120



The Alchemist

- pages = 192



Da Vinci Code

- pages = 400

borrows

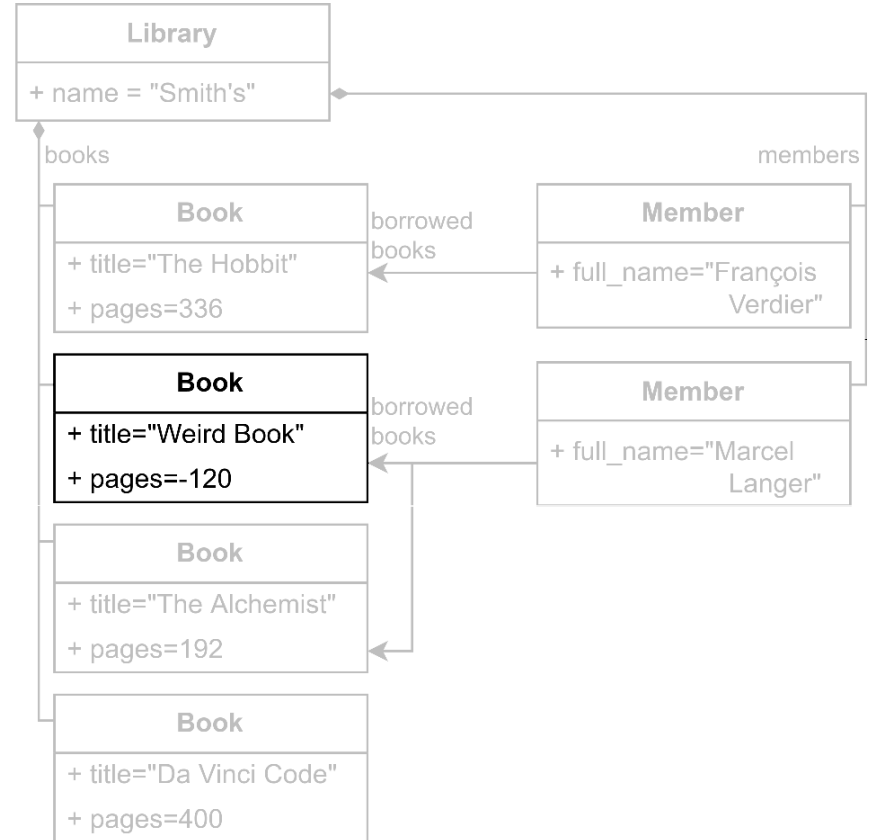


François Verdier

borrows



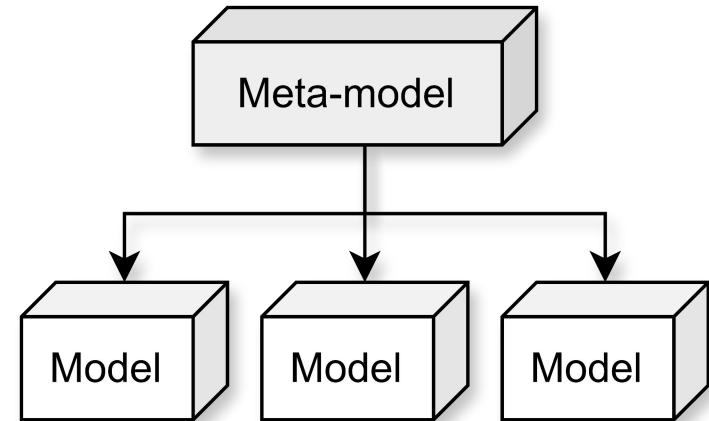
Marcel Langer



What is the Object Constraint Language ?

OCL: Object Constraint Language

A formal language describing rules, constraints and queries applying to UML models.



Fixing the invalid model example:

A **Book** must have a positive number of **pages**

context **Book**:

inv positive_pages: self.**pages** > 0



The Hobbit

- pages = 336



Weird Book

- pages = -120



The Alchemist

- pages = 192



Da Vinci Code

- pages = 400



Book

+ title="The Hobbit"

+ pages=336



Book

+ title="Weird Book"

+ pages=-120



Book

+ title="The Alchemist"

+ pages=192



Book

+ title="Da Vinci Code"

+ pages=400



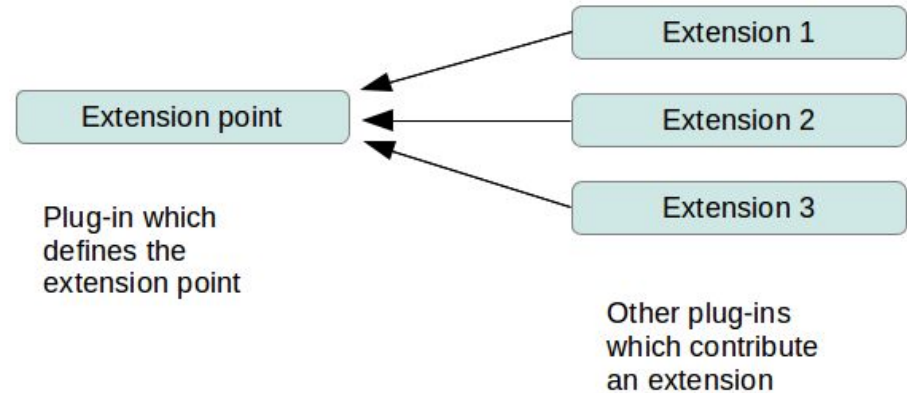
What is Eclipse ?

Popular platform for MBSE and software development (eg. IDEs) :

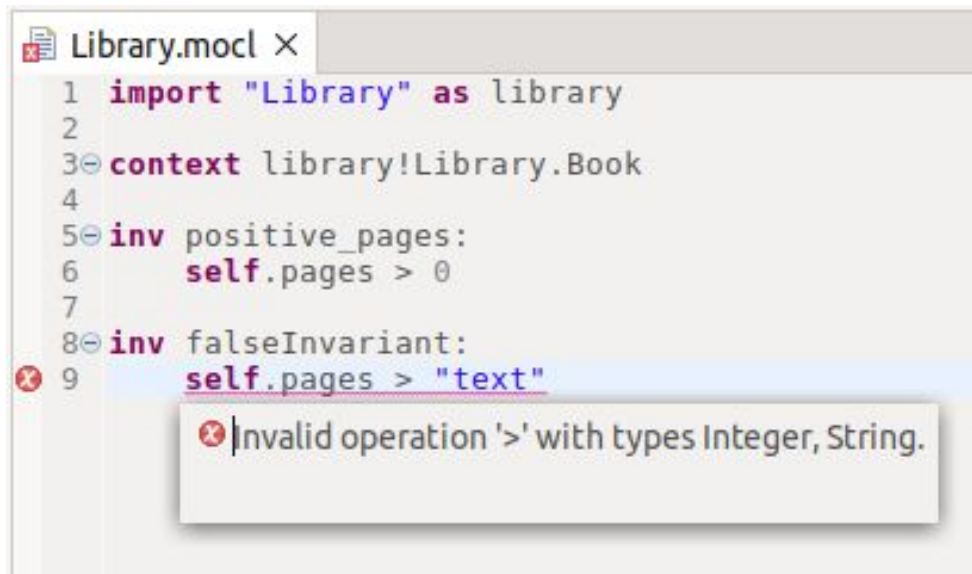
- Used at N7
- Open source
- Modular



Plugin



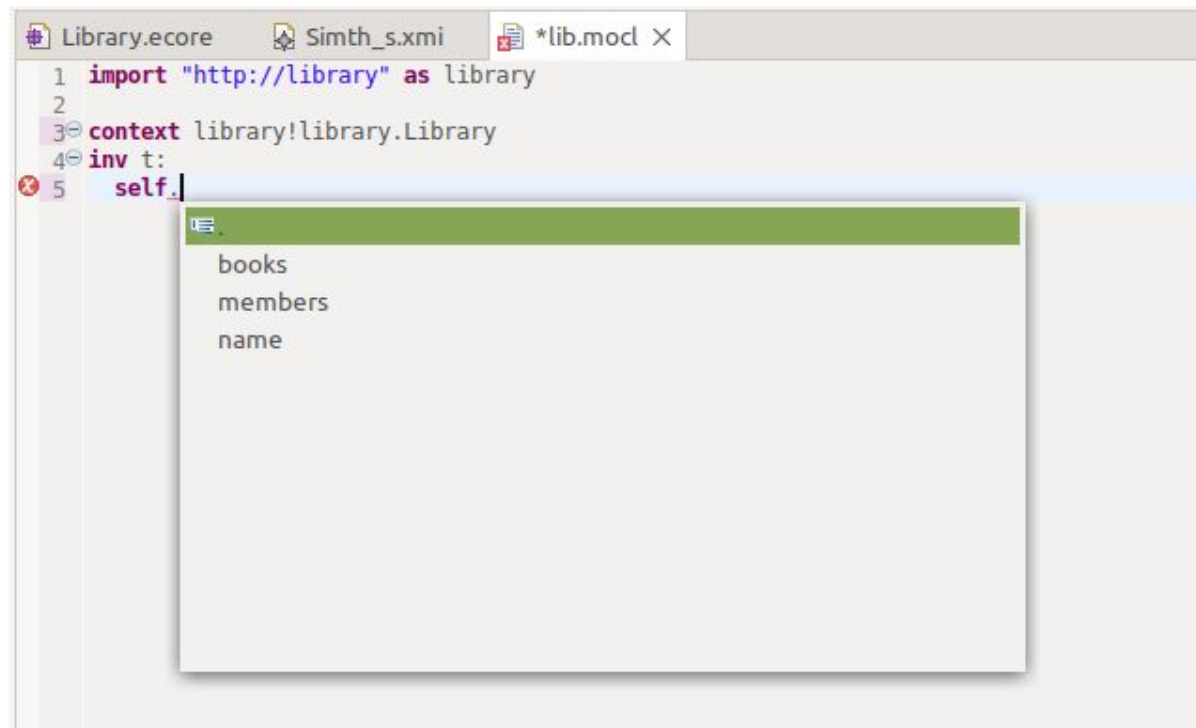
OCL in Eclipse



Example of an OCL file in editor

Autocomplete

Help students for OCL development



The screenshot shows an OCL editor window with three tabs: 'Library.ecore', 'Simth_s.xmi', and '*lib.modl X'. The code in the editor is as follows:

```

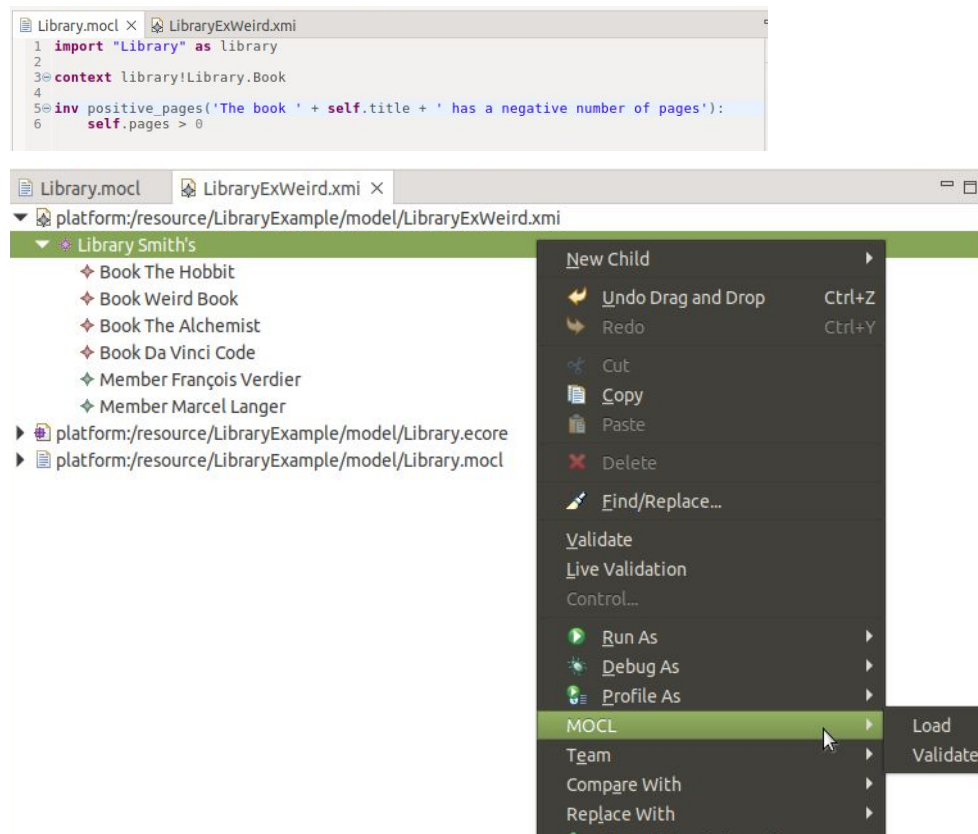
1 import "http://library" as library
2
3 context library!library.Library
4 inv t:
5   self.

```

An autocomplete dropdown menu is open below the 'self.' line, displaying the following suggestions:

- books
- members
- name

Workflow



```

Library.mocl x LibraryExWeird.xmi
1 import "Library" as library
2
3 context library!Library.Book
4
5 inv positive_pages('The book ' + self.title + ' has a negative number of pages'):
6     self.pages > 0
    
```

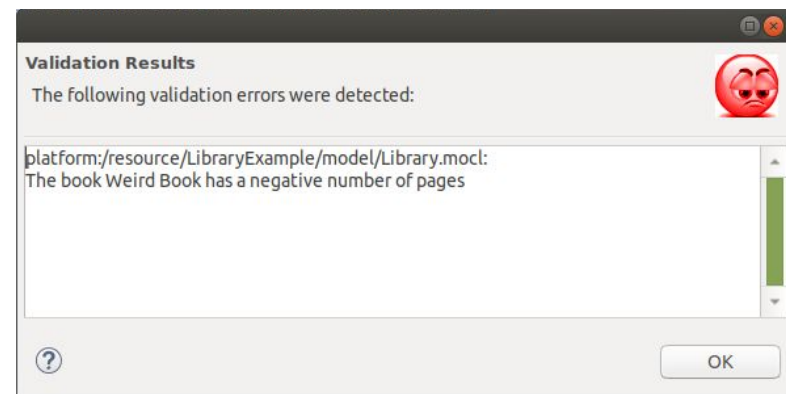
Library.mocl x LibraryExWeird.xmi

platform:/resource/LibraryExample/model/LibraryExWeird.xmi

- Library Smith's
 - Book The Hobbit
 - Book Weird Book
 - Book The Alchemist
 - Book Da Vinci Code
 - Member François Verdier
 - Member Marcel Langer
- platform:/resource/LibraryExample/model/Library.ecore
- platform:/resource/LibraryExample/model/Library.mocl

Context Menu:

- New Child
- Undo Drag and Drop (Ctrl+Z)
- Redo (Ctrl+Y)
- Cut
- Copy
- Paste
- Delete
- Find/Replace...
- Validate
- Live Validation
- Control...
- Run As
- Debug As
- Profile As
- MOCL**
 - Load
 - Validate
- Team
- Compare With
- Replace With

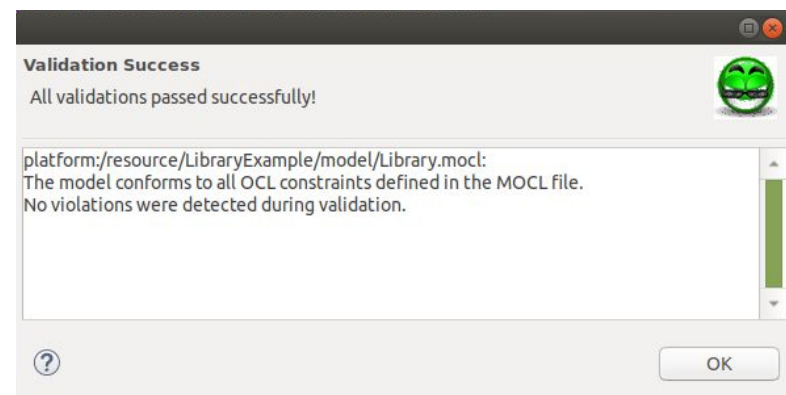


Validation Results

The following validation errors were detected:

platform:/resource/LibraryExample/model/Library.mocl:
The book Weird Book has a negative number of pages

OK



Validation Success

All validations passed successfully!

platform:/resource/LibraryExample/model/Library.mocl:
The model conforms to all OCL constraints defined in the MOCL file.
No violations were detected during validation.

OK

Main objectives

- Replace the existing (broken) OCL plugin with a new one that :
 - Can evaluate and validate OCL
 - Guide the student with comprehensive errors (validation, types)

eclipse-ocl/**org.eclipse.ocl**



Preliminary work

Main conclusions :

- List of essential features
- Understanding the compiler (original project)
- Use our last year's schoolwork as targets
- Three independent tasks
 - Type checking
 - Interpreter
 - Eclipse plug-in

Flaws of the original project



No documentation



Merge of many projects with different goals



Compiler architecture very specific and totally incompatible with interpreter
(= re do everything!)

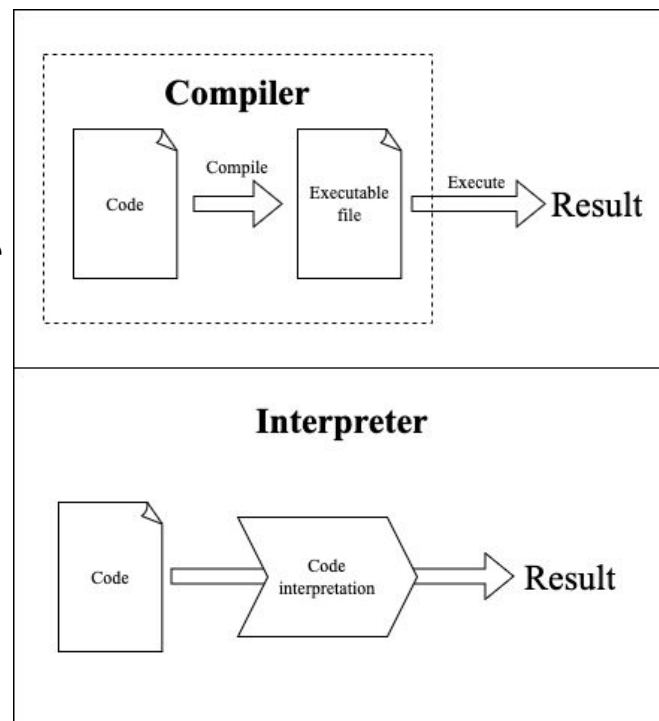
What is the difference between Compiler & Interpreter ?

Compiler :

- Faster after compilation
- Need to generate a new file for each code

Interpreter :

- Need to read the code each time
- Faster at first run



Focus on the useful

Choices we made :



Not wasting time on maintaining the compiler



Make an expendable foundation rather than focus on details



Write a comprehensive documentation to help maintenance and evolution

Working as a Team

Team work tools used in this project:

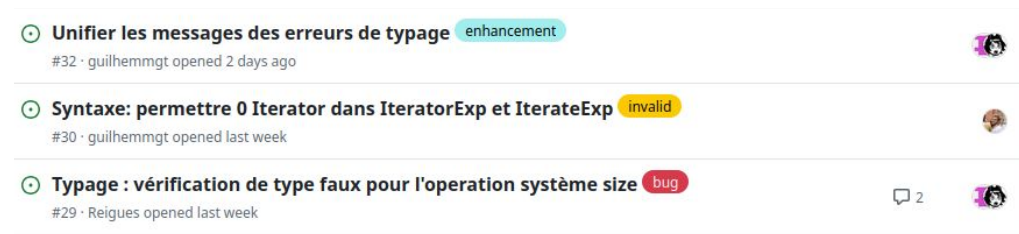
- Git and GitHub
- Agile (SCRUM) framework
- Precise work packages

GitHub : powerful tool for team coding



Powerful version control tool :

- Simplifies cooperation
- Tracks goals and issues
- Perfect for future maintenance



Examples of issues

Daily meeting (20min avg.)



Present our work to the rest of the team

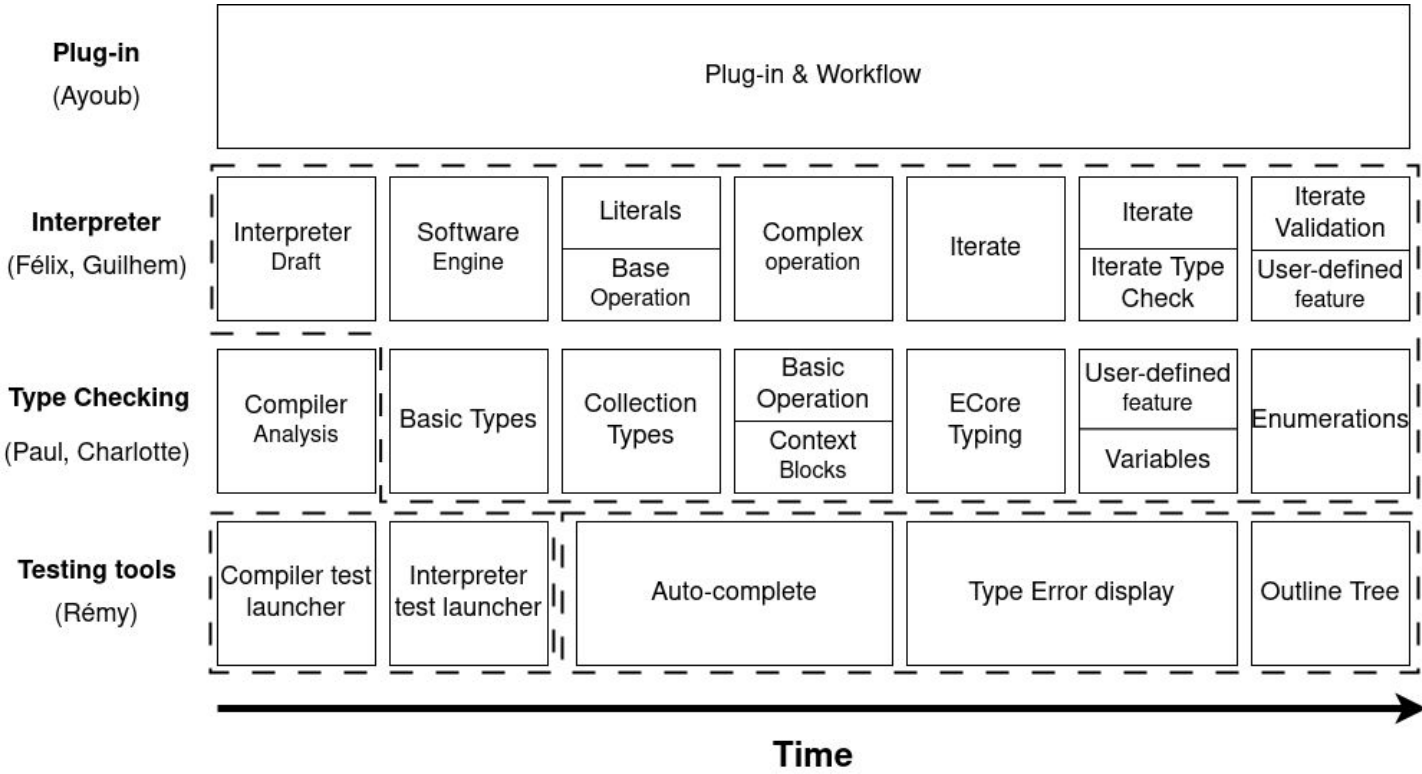


Highlight difficulties



Adjust goals to avoid high dependencies between work packages

Organizing the team's work



Test Driven Development

- Every time a new feature needed to be implemented, multiple tests were created around this feature.
- The tests help the implementation.
- For instance :

inv int:
1 + 1 = 2

inv falseInt:
1 + 1 = 3

Total number of test files : 176

The screenshot shows the JUnit 5 test runner interface. At the top, it displays the status: 'Runs: 176/176', 'Errors: 3', and 'Failures: 1'. Below this, a tree view shows the execution of all tests. The 'TestsUnitaires' section is expanded, showing a list of tests including 'falseInv', 'OperatorCallExp7', 'OperatorCallExp5', 'AddOpCallExp', 'OperatorCallExp4', 'OperatorCallExp1', 'OperatorCallExp3', 'OperatorCallExp2', 'OperatorCallExp6', 'IfExp', 'def', 'emptyXmiFalseInv', 'AttributManquant', 'accesAttributChaine', 'accesAttribut', 'msgExample', 'enum', and 'collectionEClassSet'. The 'def' test is highlighted in green, indicating it passed. At the bottom, a 'Failure Trace' section shows an error: 'java.lang.AssertionError: Valeur "invalid" retournée. Cannot access to : .defclassuse because it is null'.

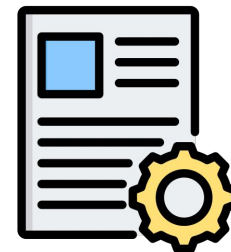
Docs



Documentation

Check Type
Operation
Iteration
JET et JMerge
Xtext Editor
Examples

Docs



For each category :

- Presentation of the category
- How to use it
- How to improve it

Documentation

{ Check Type
Operation
Iteration
JET et JMerge
Xtext Editor
Examples

Docs

All features that can be implemented are attached with precise examples to help the improvement of the project

Ajouter une opération de base

Pour ajouter une nouvelle opération il faut créer une nouvelle classe qui implémente l'interface *IOclOperation*, et la rajouter dans l'enum *OclOperationEnum*.

Imaginons que nous devons ajouter une nouvelle operation.

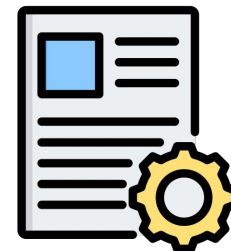
1. Créer la classe *OclNomOperation*

```
public class OclNomOperation implements IOclOperation {  
  
    @Override  
    public Object getReturnValue(Object source, List<Object> args, EObject conte  
    // Calcul de la valeur de retour  
        return ((String) source).length();  
    }  
}
```

2. Ajouter la classe à *OclOperationEnum*

Ouvrir la classe *OclOperationEnum* et y ajouter la classe au classe coresspondant à l'operation size

```
size(Arrays.asList(new OclSize(), new OclSizeString()),
```



Documentation

Check Type
Operation
Iteration
JET et JMerge
Xtext Editor
Examples



Completion Table

	Types	Operations	Iterators	Definitions	Flow of control	References
Syntax	AC	C	C	C	C	C
Type checking	AC	AC	AC	C	C	C
Autocomplete	C	U	U	C	C	C
Interpreter	AC	AC	AC	C	C	C

- C : Complete

- P : in Progress

- U : Unimplemented

- AC : Almost Complete

- S : Started

The Goals and Achievements of the Project

- Functional : Basic version of OCL that works.
- Educational : Easy to use for 2nd year students.
- Future proof : Easy to modify and expand upon.

Results :

- All the unit tests pass.
- Some student models are already verifiable.
- Auto-completion is still in progress.
- Expendable structure & comprehensive documentation
⇒ All the operations and iterators left are easy to add.

Beyond the project

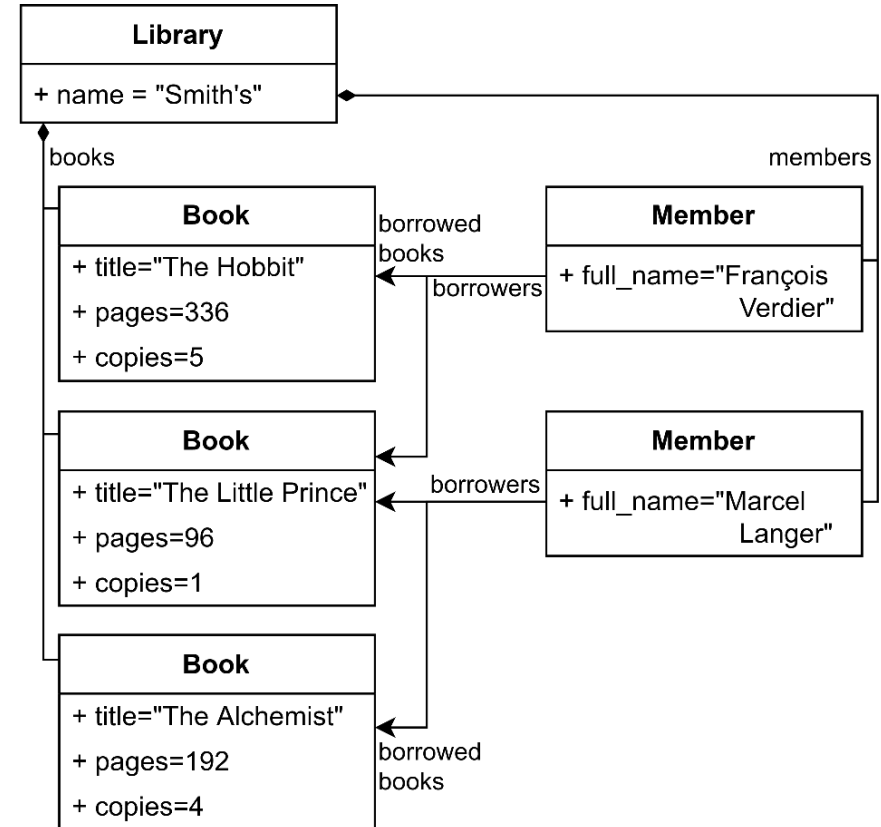
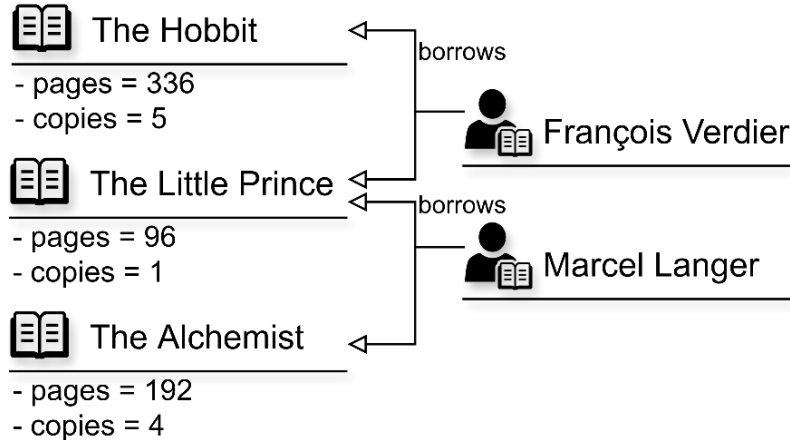
- Most goals were met.
- Can always be improved.
- To expand the scope :
 - Add more complex types
 - Add model operations
 - Example : an operation that lets a person borrow a book in the model
 - Useful for conditions before and after the operation takes place.
 - Add the more specialized operations.

Questions ?

Ayoub CANON, Félix HORCHOLLE, Charlotte LADRIERE
Guilhem MANGENOT, Rémy SANGOI, Paul SOMSON

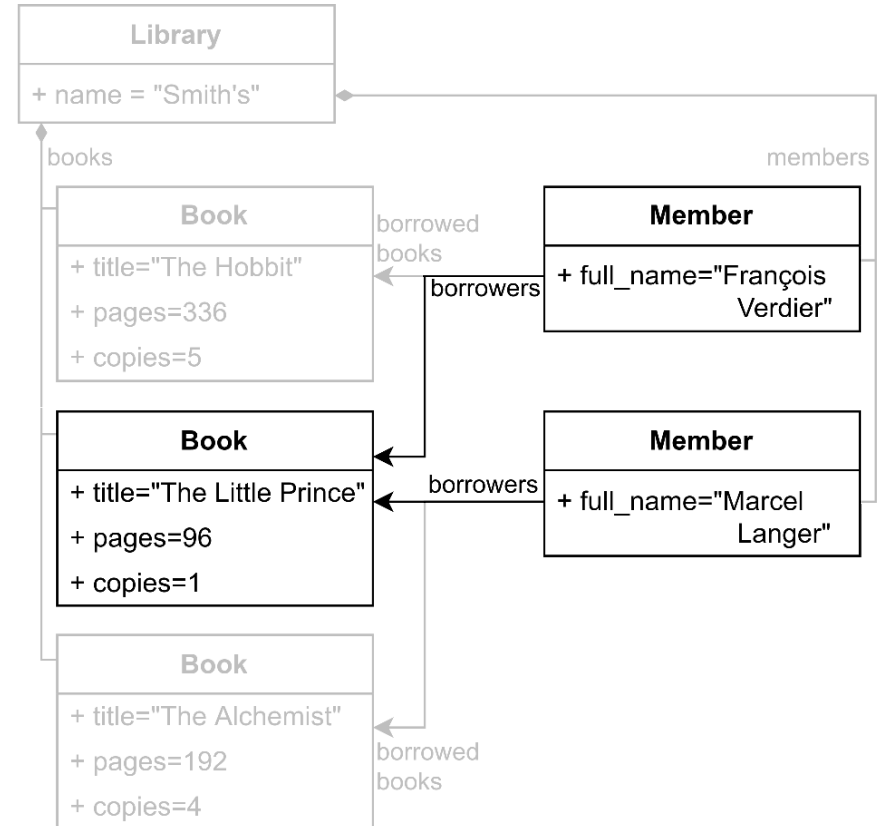
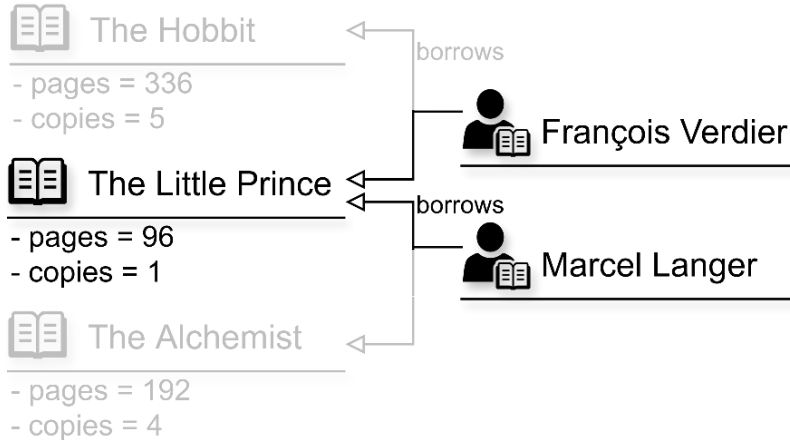
Supervisors : Xavier Cregut & Guillaume Dupont

Another invalid example:



<https://www.omg.org/spec/OCL/2.4/PDF>
<https://download.eclipse.org/ocl/doc/5.0.0/ocl.pdf>

Another invalid example:



Fixing the invalid model example:

A **Book** cannot have more **borrowers** than it has **copies**

context **Book**:

inv borrow: self.**borrowers**.size() ≤ self.**copies**



- pages = 336
- copies = 5



- pages = 96
- copies = 1



- pages = 192
- copies = 4

